Portable Spec Kit

AI Agentic Specification-Driven Development

A single portable file that defines your complete development methodology — personalized from the first session. Drop it into any project and the AI agent detects your GitHub profile, adapts to your expertise and working style, follows your engineering standards, manages tasks, writes tests, and maintains context across sessions.

SPECS → PLANNING → TASKS → BUILD + TEST → TRACK

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Ph.D. Computer Science — Artificial Intelligence

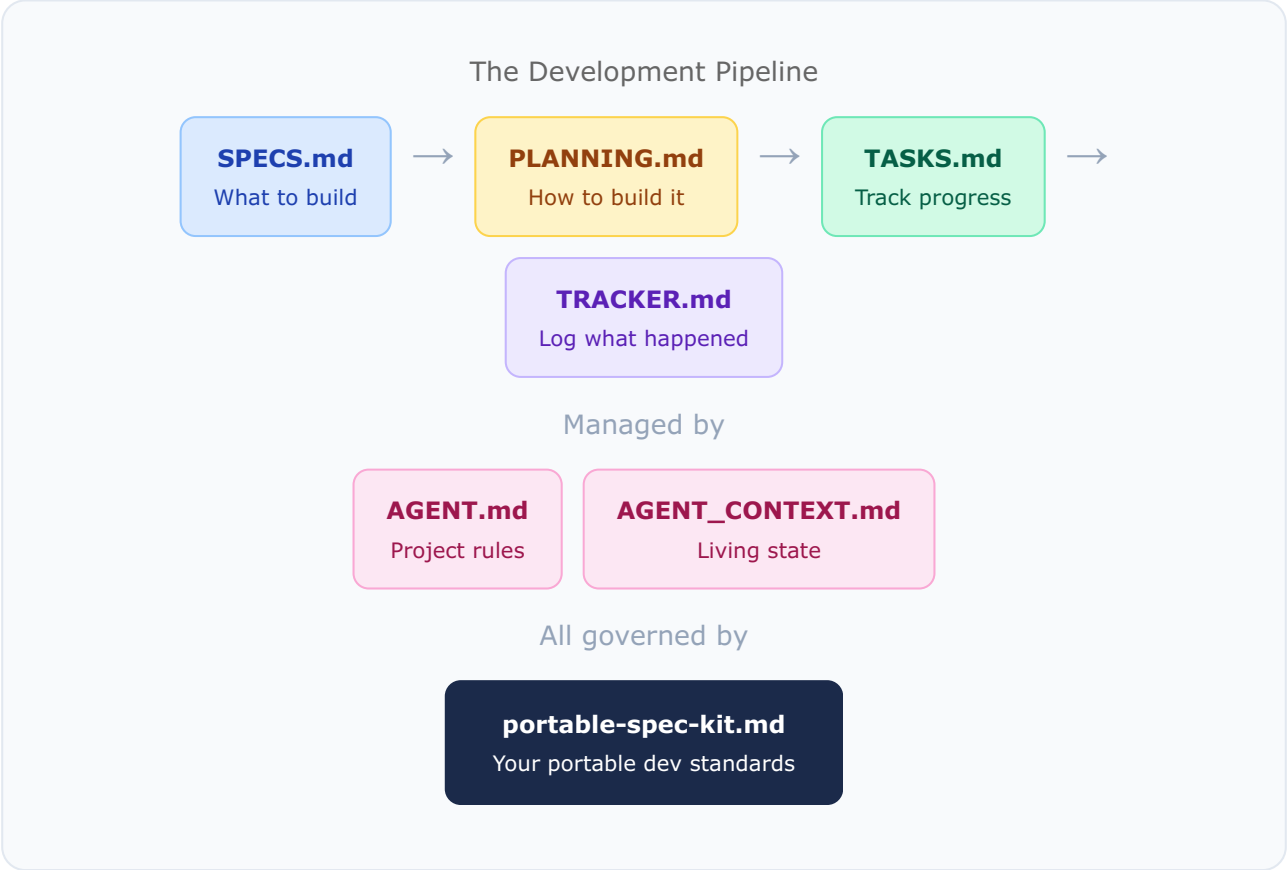
Research: Multimodal AI, Healthcare AI, Autonomous Surveillance

Document Version: 1.0.0

Date: March 2026

The Framework

The **Spec-Driven Development Framework** is a methodology where every project follows a structured pipeline from requirements to deployment, managed by an AI agent that learns and follows your engineering practices.



10 Core Strengths

Strength	What It Means
🪶 **Lightweight**	Single markdown file. Zero dependencies. Zero install.
📦 **Portable**	One file → any repo. Works instantly. Symlinks for all agents.
👤 **Personalized**	GitHub profile auto-detect. Adapts to your expertise, communication style, and working pattern. Tailored AI behavior.
📋 **Spec-Driven**	SPECS → PLAN → TASKS → TRACK. Complete dev pipeline. Retroactive spec-filling.
	Claude, Copilot, Cursor, Windsurf, Cline — one source, all sync.

🤖 **Agent-Agnostic**	
🧠 **Context-Persistent**	AGENT_CONTEXT.md carries state across sessions. Pick up after weeks. Never lose context.
🔒 **Secure**	Never expose API keys — even if asked. Strict .env rules. Code security enforced.
🔄 **Non-Blocking**	Code first, specs later. Agent tracks silently. Never blocks the developer.
🏗️ **Self-Scaffolding**	Auto-creates 6 agent files, README, workspace context. 8 stack templates built-in. Ready to code in seconds.
✅ **Self-Validating**	Agent tests everything. Fixes before presenting. 98%+ coverage target.

What the Framework File Contains

The `portable-spec-kit.md` file is the single source of truth for your development methodology. It contains:

Section	What It Governs
User Profile	Personalized AI — GitHub auto-detect, communication style, working pattern, AI delegation
Git & GitHub Rules	When to commit, push, ask before critical ops
Security Rules	Secret handling, .env practices, code security
Versioning	v0.1 → v1.0 numbering, changelog standards
Task Tracking	Add tasks FIRST, then work. Module-based organization.
Testing Standards	Coverage targets, edge case checklist, mock rules, self-validation
Code Quality	Review checklist, naming conventions, deployment checklist
Error Handling	Structured errors, logging, error boundaries, user-friendly messages
Branch & PR Workflow	Feature branches, PR format, squash merge, clean history

Dependencies	Bundle size checks, lock files, audit, avoid unnecessary deps
Project Templates	6 agent files + README + 8 source code structures (Web, Python, Mobile, Android, iOS, Full Stack, Full Stack + Mobile, Document)
Auto-Scan	Detects existing projects, creates missing files, restructures to standard
Agent Behavior	Guide don't enforce, silent tracking, retroactive spec-filling

Multi-Agent Support

One source file — `portable-spec-kit.md` — works with every AI coding agent via symlinks:

Agent	File Created	How
Claude Code	`CLAUDE.md`	Symlink → portable-spec-kit.md
GitHub Copilot	`.github/copilot-instructions.md`	Symlink → portable-spec-kit.md
Cursor	`.cursorrules`	Symlink → portable-spec-kit.md
Windsurf	`.windsurfrules`	Symlink → portable-spec-kit.md
Cline	`.clinerules`	Symlink → portable-spec-kit.md

macOS/Linux: Uses symlinks — edit `portable-spec-kit.md`, all agents read the update instantly.

Windows: Uses copies — re-run install command after editing, or let the AI agent keep files in sync.

Complete Flow

1. **Install** — one command downloads `portable-spec-kit.md` + creates symlinks for all agents (Claude, Copilot, Cursor, Windsurf, Cline)

↓

2. **Open any AI agent — auto-scaffolds** — detects environment, creates WORKSPACE_CONTEXT.md, creates `agent/` with 6 files (AGENT, SPECS, PLANNING, TASKS, TRACKER, CONTEXT), project dirs (src/, tests/, docs/), README, .gitignore, .env.example

↓

3. **Work however you want** — "build me X" → agent adds to TASKS.md, builds, tests. "What should I do next?" → agent walks through spec-driven flow. Never blocks.

↓

4. **Session ends** — agent updates AGENT_CONTEXT.md with progress, decisions, what's next

↓

5. **Come back later** — days, weeks, months. Agent reads AGENT_CONTEXT.md: "Here's where we left off." Continues exactly where you stopped.

↓

6. **Edit the framework** — update portable-spec-kit.md, all 5 agent symlinks read the change instantly. Standards evolve with you.

Project Structure

```
your-project/
├── portable-spec-kit.md    ← Framework file (source)
├── .portable-spec-kit/     ← Kit config (user profiles, per-user)
│   └── user-profile/
│       └── user-profile-{username}.md
├── CLAUDE.md               ← Symlink (Claude Code)
├── .cursorrules            ← Symlink (Cursor)
├── WORKSPACE_CONTEXT.md    ← Auto-created (workspace state)
├── README.md               ← Auto-created (standard structure)
│
├── agent/                 ← Auto-created (project management)
│   ├── AGENT.md           ← Project rules, stack, brand
```

```
│   ├── AGENT_CONTEXT.md   ← Living state (updated every session)
│   ├── SPECS.md           ← Requirements & features
│   ├── PLANNING.md        ← Architecture, methodology & research
│   ├── TASKS.md           ← Task tracking (module-based)
│   └── TRACKER.md         ← Version history
│
├── src/                   ← Your code
├── tests/                 ← Your tests
├── ard/                   ← Architecture docs (HTML + PDF)
└── ...
```

The 6 Agent Files

File	Purpose	When
`AGENT.md`	Project-specific instructions — stack, rules, brand, AI config	Setup
`AGENT_CONTEXT.md`	Living state — done, next, decisions, blockers, file structure	Every session
`SPECS.md`	What to build — requirements, features, acceptance criteria, scope	Before dev
`PLANNING.md`	How to build — architecture, data model, phases, methodology, research, decision log	Before dev
`TASKS.md`	Module-based task tracking with progress summary table	During dev
`TRACKER.md`	Version changelog (categorized), test results, deployment log	End of version

How to Work in Spec-Driven Development

Phase 1: Specification

> **Goal:** Define WHAT you're building before touching code.

Start every project by writing `agent/SPECS.md`. This is a conversation between you and the AI:

You Say	AI Does

"I want to build a resume editor"	Writes requirements, features table, scope boundaries, acceptance criteria in SPECS.md
"Add AI-powered bullet rewriting"	Adds feature to specs, estimates priority, updates scope
"That's out of scope for v1"	Moves to "Out of scope (future)" section

Rule: No code is written until SPECS.md has requirements and acceptance criteria.

Phase 2: Planning

> **Goal:** Define HOW you're building it — architecture, data model, phases.

AI writes `agent/PLANNING.md` based on approved specs:

Section	What AI Fills In
Stack	Recommends tech (frontend, backend, DB, hosting) with trade-offs. You approve.
Architecture	System design, component relationships, data flow
Directory Structure	Exact file/folder layout for the chosen stack
Data Model	Tables, types, relationships
API Endpoints	Method, path, description for each route
Build Phases	Phased delivery plan — foundation first, features later
Security	Auth, CORS, input validation, secret handling

Rule: Deployment is deferred to release time. Don't discuss hosting during planning.

Phase 3: Execution

> **Goal:** Build, test, and track every task.

AI creates `agent/TASKS.md` with module-based breakdown:

```
## Module 1: Project Setup
| # | Task | Status |
```

```
|---|------|:------:|
| 1.1 | Initialize Next.js + Tailwind | [x] |
| 1.2 | Create component structure | [ ] |

## Module 2: Core Features
| # | Task | Status |
|---|------|:------:|
| 2.1 | Build editor canvas | [ ] |
```

Execution rules enforced by the framework:

Rule	What It Means
Tasks first	When you ask for something, AI adds it to TASKS.md BEFORE starting work
Self-validate	AI runs tests, fixes failures, only presents stable results
Test everything	Every feature gets tests — unit, integration, UI interaction
Edge cases	Empty inputs, boundaries, XSS, unicode, error paths — all tested
Coverage targets	Backend 98%+, frontend logic 98%+, UI components 85%+
No broken features	User should NEVER discover something doesn't work
Context updated	AGENT_CONTEXT.md updated after every implementation + test run, and at session end
Flows updated	docs/flows/ updated if any system flow changed — flows always match code

Phase 4: Tracking

> **Goal:** Log what was delivered, tested, and deployed.

At end of each version, AI updates `agent/TRACKER.md`:

```
## v0.2 — AI Features (March 30, 2026)

### Summary
Added JD-tailored resume, PDF generation, 604 tests.

### Changes
- **Frontend:** JD Tailor modal, version manager, skills tags
```

```
- **Backend:** 3 PDF engines, expand API, fetch-url API
- **AI:** Bullet format detection, deduplication, length matching

### Tests
- 604 tests passing, 95% coverage

### Deployment
- Deployed to: aqibmumtaz.github.io
- Date: 2026-03-30
```

Daily Workflow Summary

1. **Start session** — AI reads agent/AGENT.md + AGENT_CONTEXT.md. Knows exactly where you left off.

↓

2. **You give instructions** — "add dark mode", "fix scoring bug", "build PDF export"

↓

3. **AI adds to TASKS.md** first, then builds, tests, and shows results with coverage

↓

4. **You review** — approve, adjust, or ask for changes. AI fixes and re-tests.

↓

5. **You say "commit"** — AI commits with descriptive message. "push" sends to remote.

↓

6. **Session ends** — AI updates AGENT_CONTEXT.md. Next session picks up seamlessly.

> **The result:** Every project follows the same spec → plan → build → test → track pipeline. Nothing is ad-hoc. Nothing falls through cracks. Context never lost.

Agent Behavior — Guide, Don't Enforce

The framework is **not rigid**. The AI agent adapts to how you want to work:

> **Core principle:** The user can start anywhere — jump into coding, give direct tasks, ask questions, or follow the full spec-driven flow. All valid. The agent adapts to the user, not the other way around.

You Do	Agent Does
"Build me a login page"	Adds to TASKS.md → builds it → tests it → shows results
"Fix this bug"	Adds to TASKS.md → fixes → marks done
"What should I do next?"	Walks through spec-driven process: define specs → plan → tasks
"What's the status?"	Shows from TASKS.md + AGENT_CONTEXT.md
Come back after weeks	Reads AGENT_CONTEXT.md → summarizes where you left off
Never wrote specs	Retroactively fills SPECS.md from what's built — no complaints

Silent Tracking

Even if you never mention the process, the agent:

- Adds every task to TASKS.md before starting
- Marks tasks done when complete
- Updates AGENT_CONTEXT.md at session end
- Fills gaps in SPECS.md and PLANNING.md retroactively
- Surfaces process naturally: "I've added this to TASKS.md", "Based on SPECS.md, we still have..."

> **User's time is sacred.** Agent does 90% of the work — writes specs, plans, tasks, tests, docs. User reviews 10%. Never ask the user to write specs or plans — the agent drafts everything, user approves or adjusts.

File Creation/Update Rule

This single rule governs how the agent handles ALL managed files (`WORKSPACE_CONTEXT.md`, `README.md`, and all `agent/` files):

Scenario	Action
File does not exist	Create from standard template, fill in known details
File exists but wrong structure	Restructure to match template — **retain all existing content**, never lose data
File already matches template	Leave as-is

> **Never lose content.** When restructuring an existing file, every detail, decision, and note is preserved — just reorganized into the standard sections.

Customization

User Profile — Personalized AI Experience

On first session, the agent fetches your GitHub identity and asks 3 quick questions. Press Enter to use recommended, or type your own:

Question	Options
Communication style?	(a) direct and concise ← RECOMMENDED (b) direct, data-driven, comprehensive with tables (c) conversational and collaborative Press Enter to use recommended (a)
Working pattern?	(a) iterative — start small, expand ambitiously ← RECOMMENDED (b) plan-first — full specs before code (c) prototype-fast — build quick, refine later Press Enter to use recommended (a)
AI delegation?	(a) AI does 70%, user guides 30% ← RECOMMENDED (b) AI does 90%, user reviews 10% (c) 50/50 collaboration Press Enter to use recommended (a)

Saved to:

```
~/.portable-spec-kit/user-profile/user-profile-janesmith.md         (global — asked once)
workspace/.portable-spec-kit/user-profile/user-profile-janesmith.md (committed — persists)
```

> **Personalized from first session.** Profile asked once, saved globally. On new projects: shown and confirmed — keep or customize per project. Customized profiles save to workspace. Press Enter through everything for recommended defaults.

Your Projects

Project-specific details go in `agent/AGENT.md` (auto-created per project). This keeps the framework file portable while each project has its own stack, brand, and rules:

```
## Stack
| Layer | Technology |
|-------|-----------|
| Frontend | Next.js + TypeScript |
| Backend  | FastAPI |

## Brand
- Primary: #1B2A4A
- Accent: #4a6fa5
```

Dev Stack Adaptation

The framework uses stack-agnostic language for testing, building, and linting. Each project's `agent/AGENT.md` defines the actual commands:

Framework Says	Project Defines
"All tests passing"	`npm test` / `pytest` / `go test`
"Type checking: zero errors"	`npx tsc --noEmit` / `mypy`
"Linting: zero errors"	`eslint` / `ruff` / `golint`
"Build succeeds"	`npm run build` / `docker build`
"PDF generation tool"	WeasyPrint / Puppeteer / wkhtmltopdf

> **Stack-agnostic by design.** The framework never assumes a specific language, build tool, or test runner. It defines *what* to check — each project defines *how*.

Sharing & Syncing the Framework

Recommended: Dedicated GitHub Repo

📦 Single-Source-of-Truth Repo

One repo holds portable-spec-kit.md. All projects pull from it. Updates pushed back from any project.

```
# Setup (once)
mkdir portable-spec-kit && cd portable-spec-kit
git init && cp /path/to/portable-spec-kit.md .
git add . && git commit -m "Framework v1.0"
gh repo create portable-spec-kit --private --push

# Pull into any project
curl -sO https://raw.githubusercontent.com/you/portable-spec-kit/main/portable-spec-kit.md

# Push updates back from any project
cp portable-spec-kit.md ~/portable-spec-kit/
cd ~/portable-spec-kit && git add . && git commit -m "Updated from ProjectX" && git push
```

✓ Version controlled ✓ Pull anywhere ✓ Change history ✓ Cross-machine sync

Other Options

Method	How	Best For
Git Submodule	`git submodule add repo .framework`	Auto-sync across repos
GitHub Gist	`gh gist create portable-spec-kit.md`	Quick sharing, browser editable
Shell Alias	`alias init-dev='curl -sO URL/portable-spec-kit.md'`	One-command setup

Update Workflow Across Projects

1. **Discover new practice** while working on Project A (e.g., new testing rule)

↓

2. **AI updates portable-spec-kit.md** in Project A with the new rule

↓

3. **Push to shared repo** — single source of truth updated

↓

4. **Pull in Project B** — instantly has the new rule. AI follows it immediately.

Portable vs Project-Specific

In portable-spec-kit.md (Portable Framework)	In agent/AGENT.md (Per Project)
Git/push/commit rules	Tech stack choices
Security practices	Brand colors & fonts
Testing methodology	AI model/provider config
Naming conventions	Dev server port
Project templates & auto-scan	Deployment specifics
Code review standards	Project-specific rules
Spec-driven development flow	API endpoints & data model
Version numbering scheme	Database schema

Examples Included

The repo includes two example projects showing the framework in action:

Example	What It Shows	Best For
`examples/starter/`	Fresh project right after setup. Self-documenting README explains every file and directory.	Understanding the framework — **start here**
`examples/my-app/`	Mid-development Next.js + Supabase project. 11/16 tasks done, 24 tests at 92% coverage.	Seeing a realistic project in progress

Starter Project Structure

```
examples/starter/
├── portable-spec-kit.md    ← Framework file (source)
├── .portable-spec-kit/     ← Kit config (user profiles)
│   └── user-profile/
├── CLAUDE.md               ← Symlink (Claude Code)
├── .cursorrules            ← Symlink (Cursor)
├── WORKSPACE_CONTEXT.md    ← Auto-created workspace state
├── README.md               ← Self-documenting (explains everything)
├── agent/
│   ├── AGENT.md           ← Stack: TBD (waiting for your specs)
│   ├── AGENT_CONTEXT.md   ← Status: "Setup — waiting for specs"
│   ├── SPECS.md           ← Empty — ready for requirements
│   ├── PLANNING.md        ← Empty — ready for architecture & methodology
│   ├── TASKS.md           ← 1/5 tasks (project initialized)
│   └── TRACKER.md         ← v0.1 placeholder
```

Mid-Development Project

```
examples/my-app/
├── agent/
│   ├── AGENT.md           ← Next.js + Supabase configured
│   ├── AGENT_CONTEXT.md   ← v0.1: 11/16 tasks, 24 tests
│   ├── SPECS.md           ← 8 features, priorities, criteria
│   ├── PLANNING.md        ← Data model, APIs, 3 phases
│   ├── TASKS.md           ← 5 modules with progress
│   └── TRACKER.md         ← Categorized changelog
```

> **One file. Any project. Your standards.**
> The framework travels with you — consistent engineering practices across every project, every team, every AI session.

Portable Spec Kit

Thank You

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Ph.D. Computer Science — Artificial Intelligence

Research: Multimodal AI, Healthcare AI, Autonomous Surveillance

Email: aqib.mumtaz@gmail.com

LinkedIn: linkedin.com/in/aqibmumtaz

GitHub: github.com/aqibmumtaz

Document Version 1.0.0 • March 2026

© 2026 Dr. Aqib Mumtaz. All rights reserved.